CIESIELSKI THOMAS
2SIO2

SERVEUR WEB ET HTTPS

I. Configuration du serveur Web

- Configuration du réseau

Pour configurer l'adresse réseau de notre serveur Web il faut effectuer la commande **nano /etc/network/interfaces**, l'adresse réseau de notre serveur sera configuré de la sorte :

```
  GNU nano 8.4                                                    /etc/network/interfaces *
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5)

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug ens18
iface ens18 inet static
address 172.16.144.100
netmask 255.255.0.0
gateway 172.16.0.1
dns-nameserver 172.16.0.100
```

Remarque : l'interface réseau que l'on configure est « ens18 ».

Après avoir redémarrer le serveur, effectuez la commande **ip a** afin de vérifier l'adressage IP :

```
root@Debian:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:cf:dc:d8 brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    altname enxbc2411cfdcd8
    inet 172.16.144.100/16 brd 172.16.255.255 scope global ens18
       valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fecf:dcd8/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

Remarque : l'adressage IP correspond bien à l'interface « ens18 ».

Ensuite, pour tester son bon fonctionnement, on peut tester de ping la passerelle ou le DNS de Google (8.8.8.8) par exemple :

```
root@Debian:~# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=12.2 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=12.7 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=12.0 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=13.3 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=117 time=13.2 ms
```

- Sources.list

Sources.list est un document texte se situant dans /etc/apt/sources.list faisant un lien vers les dépôts afin de télécharger depuis APT des mises à jour, des paquets…

A la création de notre serveur le document sources.list doit être modifié, pour ce faire il faut exécuter les étapes suivantes :

- Effectuer la commande **su root** puis entrer le mdp

- Effectuer la commande **cd /etc/apt** (cd = change directory : change de répertoire) puis **nano sources.list** (nano : ouvre un fichier déjà existant ou le créer sinon)

- Une fois dans le fichier sources.list, il faut tout supprimer pour le remplacer par le sources.list suivant (trouvable sur wiki.debian.org) :

```
#deb cdrom:[Debian GNU/Linux 13.0.0 _Trixie_ - Official amd64 NETINST with firmware 20250809-11:20]/ trixie contrib

deb http://deb.debian.org/debian/ trixie main non-free-firmware
deb-src http://deb.debian.org/debian/ trixie main non-free-firmware

deb http://security.debian.org/debian-security trixie-security main non-free-firmware
deb-src http://security.debian.org/debian-security trixie-security main non-free-firmware

# trixie-updates, to get updates before a point release is made;
# see https://www.debian.org/doc/manuals/debian-reference/ch02.en.html#_updates_and_backports
deb http://deb.debian.org/debian/ trixie-updates main non-free-firmware
deb-src http://deb.debian.org/debian/ trixie-updates main non-free-firmware

# This system was installed using small removable media
# (e.g. netinst, live or single CD). The matching "deb cdrom"
# entries were disabled at the end of the installation process.
# For information about how to configure apt package sources,
# see the sources.list(5) manual.
```

- <u>Création des groupes et utilisateurs</u>

Les commandes suivantes permettent de créer les utilisateurs dev1 et dev2 :

su root → entrer le mdp du compte root, ici Azerty31
sudo adduser *nom utilisateur* → lui donner un mdp et entrer ses informations supplémentaires si besoin

On aura ainsi les utilisateurs suivant :

user → dev1
mdp → dev1

user → dev2
mdp → dev2

Remarque : Pour supprimer un utilisateur il suffit de taper la commande **sudo deluser *username***.

Une fois nos utilisateurs créer, nous allons les mettre dans les groupes « dev » et « sftp-users », groupes qui nous serviront plus tard. Puis mettre l'utilisateur neo dans le groupe « admin » :

sudo addgroup dev

sudo addgroup sftp-users

sudo addgroup admin

usermod -aG dev dev1

usermod -aG dev dev2

usermod -aG sftp-users dev1

usermod -aG sftp-users dev2

usermod -aG admin neo

Remarque : Pour vérifier leur appartenance à un groupe faire les commandes **groups dev1** ou **groups dev2.**

- Mise à jour des paquets

Mettre à jour les paquets se fait à l'aide des commandes : **apt-get update** puis, **apt-get upgrade**. Ils permettent de préparer le système pour la suite des opérations.

II. Installation et configuration des services

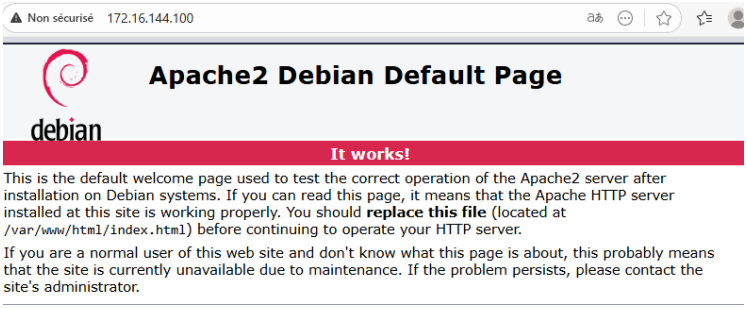
- Installation Apache

Apache est un serveur WEB permettant l'hébergement de site et application web sur internet. Son installation est extrêmement simple il suffit d'effectuer la commande : **apt-get install apache2**.

Pour tester qu'Apache2 soit bien installé sur le serveur il faut effectuer deux étapes :
-Premièrement, effectuez la commande **systemctl status apache2**, l'état du service apache2 doit être en « running » :

```
root@Debian:~# systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Mon 2025-09-15 16:36:46 CEST; 50s ago
 Invocation: 988a468a28c3485bb4df89e32b6a5d88
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 4684 (apache2)
      Tasks: 55 (limit: 4640)
     Memory: 5.1M (peak: 5.4M)
        CPU: 243ms
     CGroup: /system.slice/apache2.service
             ├─4684 /usr/sbin/apache2 -k start
             ├─4686 /usr/sbin/apache2 -k start
             └─4687 /usr/sbin/apache2 -k start
```

- Sur un navigateur de votre PC hôte, rentrez dans l'url l'adresse IP du serveur web (172.16.144.100), le message «it works» devrait s'afficher :



- Installation PHP

Le service PHP nous permet d'intégrer un interpréteur php sur notre serveur Web, celui-ci nous permettra d'intégrer des pages .php pour notre application. Voici les commandes à rentrer pour l'installer :

apt-get install php
apt-get install libapache2-mod-php
systemctl restart apache2 (redémarrer le service apache)

Pour vérifier que php soit bien installé, effectuez la commande **php -v** :

```
root@Debian:~# php -v
PHP 8.4.11 (cli) (built: Aug  3 2025 07:32:21) (NTS)
Copyright (c) The PHP Group
Built by Debian
Zend Engine v4.4.11, Copyright (c) Zend Technologies
    with Zend OPcache v8.4.11, Copyright (c), by Zend Technologies
```

- Installation MariaDB

MariaDB est un système de gestion de base de données nous permettant d'effectuer du sql.
Pour l'installer il faut faire : **apt install mariadb-server -y**
Pour vérifier l'installation, comme pour Apache, on fait : **systemctl status mariadb**

```
root@Debian:~# systemctl status mariadb
● mariadb.service - MariaDB 11.8.3 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
     Active: active (running) since Mon 2025-09-15 17:00:14 CEST; 7min ago
 Invocation: b12093997d5748f18356f33bee4519fb
       Docs: man:mariadbd(8)
```

- Installation de PhpMyAdmin

PhpMyAdmin est une interface graphique permettant de gérer des bases de données et ainsi faciliter l'écriture des requêtes, la création de tables, les mises à jour, les droits, etc…
Pour l'installer il faut effectuer la commande **apt-get install phpmyadmin** et choisir l'option apache2.

! Configuration du mot de passe !

Nous pouvons désormais accéder à l'interface PhpMyAdmin depuis le PC hôte, il suffira de rentrer l'adresse http://172.16.144.100/phpmyadmin dans la barre de recherche.

Remarque : Si il y a une erreur lors de l'accès à PhpMyAdmin du type « Not Found », tapez sur le serveur web la commande **dpkg-reconfigure phpmyadmin** et reconfigurer les mdp.

Une fois phpMyAdmin installé, on peut s'y connecter pour importer/créer ses BDD, il suffit de faire « nouvelle base de données » lui donner un nom (pour nous « GestionFrais1 » et « GestionFrais2 » car il nous en faut 2) et d'importer le fichier de création et de valeurs dans l'onglet « import ».

Voici les étapes à effectuer sur Debian13 de la création des utilisateurs de MariaDB (2 comptes Admin + 1 compte/BDD) :

mysql -u root -p
[mot de passe pour accéder à MariaDB]

admin1 :

MariaDB [(none)]> CREATE USER 'admin1'@'localhost' IDENTIFIED BY 'Azerty31';

MariaDB [(none)]> GRANT ALL PRIVILEGES ON *.* TO 'admin1'@'localhost';

MariaDB [(none)]> GRANT GRANT OPTION ON *.* TO 'admin1'@'localhost';

MariaDB [(none)]> FLUSH PRIVILEGES;

admin2 :

MariaDB [(none)]> CREATE USER 'admin2'@'localhost' IDENTIFIED BY 'Azerty31';

MariaDB [(none)]> GRANT ALL PRIVILEGES ON *.* TO 'admin2'@'localhost';

MariaDB [(none)]> GRANT GRANT OPTION ON *.* TO 'admin2'@'localhost';

MariaDB [(none)]> FLUSH PRIVILEGES;

dev1 :

MariaDB [(none)]> CREATE USER 'dev1'@'localhost' IDENTIFIED BY 'Azerty31';

MariaDB [(none)]> GRANT SELECT, INSERT, UPDATE, DELETE ON GestionFrais1.* TO 'dev1'@'localhost';

MariaDB [(none)]> FLUSH PRIVILEGES;

dev2 :

MariaDB [(none)]> CREATE USER 'dev2'@'localhost' IDENTIFIED BY 'Azerty31';

MariaDB [(none)]> GRANT SELECT, INSERT, UPDATE, DELETE ON GestionFrais2.* TO 'dev2'@'localhost';

MariaDB [(none)]> FLUSH PRIVILEGES;

MariaDB [(none)]> exit;

Résultat sur PhpMyAdmin :

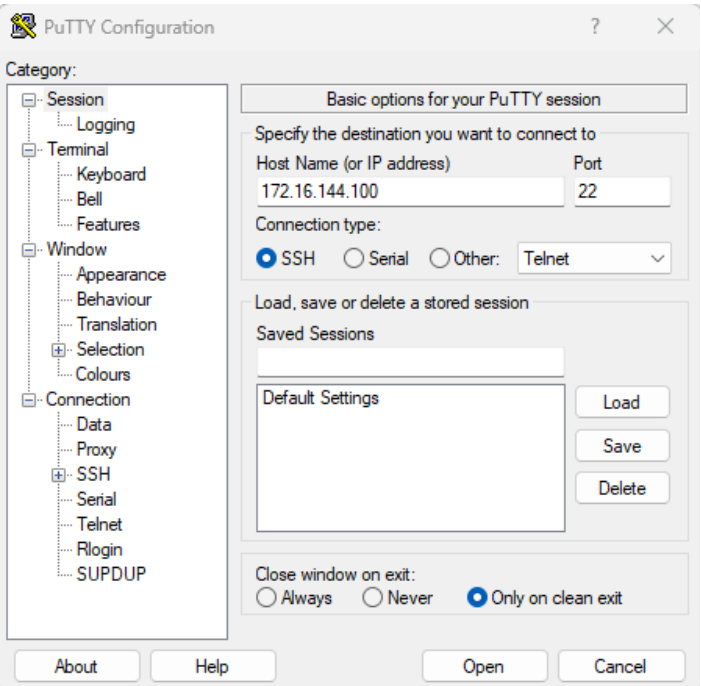
User accounts overview

	User name	Host name	Password	Global privileges	User group	Grant	Action		
☐	admin1	localhost	Yes	ALL PRIVILEGES		Yes	Edit privileges	Export	Lock
☐	admin2	localhost	Yes	ALL PRIVILEGES		Yes	Edit privileges	Export	Lock
☐	dev1	localhost	Yes	USAGE		No	Edit privileges	Export	Lock
☐	dev2	localhost	Yes	USAGE		No	Edit privileges	Export	Lock
☐	mariadb.sys	localhost	No	USAGE		No	Edit privileges	Export	Unlock
☐	mysql	localhost	Yes	ALL PRIVILEGES		Yes	Edit privileges	Export	Lock
☐	phpmyadmin	localhost	Yes	USAGE		No	Edit privileges	Export	Lock
☐	root	localhost	Yes	ALL PRIVILEGES		Yes	Edit privileges	Export	Lock

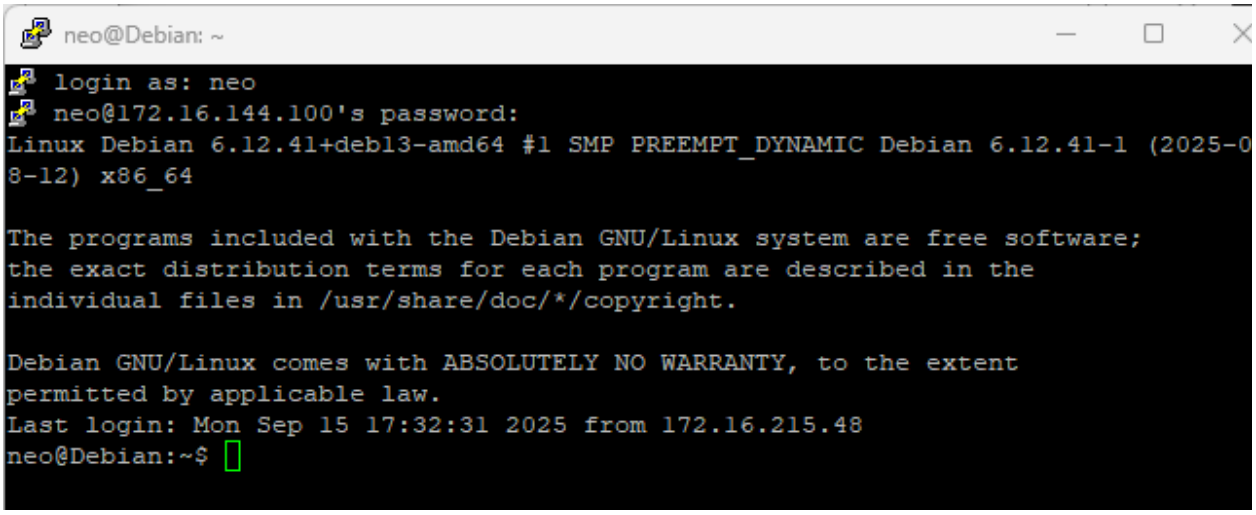
- <u>Installation SSH</u>

SSH est un protocole permettant d'établir des connexions chiffrées à distance entre ordinateurs. Ainsi, il sera possible depuis le logiciel putty d'accéder à notre serveur web grâce au protocole ssh, cela est notamment utile car il sera possible d'effectuer des copier coller. Pour l'installer il faut faire la commande **apt-get install openssh-server**.

Ensuite sur PUTTY :



Pour se connecter à notre serveur SSH, il faut renseigner dans le champ « Host Name » l'adresse IP ou le nom DNS du serveur Web et s'assurer que l'option SSH est cochée dans « Connection type », enfin cliquez sur Open.
On obtient donc un terminal dans lequel il faut renseigner le login ainsi que le password de son serveur Debian, une fois rempli on à donc accès à notre serveur à distance en ligne de commande :



Modifions maintenant le fichier sshd_config afin de restreindre le champ d'action des utilisateurs. Pour cela, effectuez la commande nano **/etc/ssh/sshd_config** et copier les lignes suivantes à la fin du fichier :

```
AllowGroups sftp-users admin
# Example of overriding settings on a per-user basis
Match User dev1
        ChrootDirectory /var/www/html/GestionFrais1
        ForceCommand internal-sftp
        X11Forwarding no
        AllowTcpForwarding no
        PermitTTY no
#       ForceCommand cvs server

Match User dev2
        ChrootDirectory /var/www/html/GestionFrais2
        ForceCommand internal-sftp
        X11Forwarding no
        AllowTcpForwarding no
        PermitTTY no
```

Ici, on autorise les membres du groupes « sftp-users » et « admin » à se connecter au serveur à distance, cependant les utilisateurs « dev1 » et « dev2 » sont isolés dans le répertoire GestionFraisX créer à l'aide de la commande **mkdir GestionFraisX** (ou X est le chiffre du site) et sont restreint à l'utilisation du sftp (« ForceCommand internal-sftp) que l'on configurera plus tard.
Le « MatchUser » pour les deux utilisateurs dev1 et dev2 fait qu'ils auront respectivement accès aux site GestionFrais1 et GestionFrais2.

Attention : Il faut modifier les droits du dossier « GestionFraisX » pour que les utilisateurs puissent y accéder depuis la commande **sudo chown root:root /var/www/html/GestionFraisX** puis **sudo chmod 755 /var/www/html/GestionFraisX**. Ensuite il faut créer un dossier « site » où l'utilisateur devX aura les droits d'écriture pour y mettre le site depuis FileZilla avec :
sudo mkdir /var/www/html/GestionFraisX/site ;
sudo chown dev1:www-data /var/www/html/GestionFraisX/site ;
sudo chmod 755 /var/www/html/GestionFraisX/site ;

- Virtual hosts et SFTP

Les hôtes virtuels permettent à Apache de gérer plusieurs arborescences Web en simultanées. Nous allons créer un virtual host lié au dossier "GestionFrais" que nous créerons plus tard

-Depuis la commande **cd /etc/apache2/sites-available**

-Créer le fichier site1.conf depuis la conf original avec **cp 000-default.conf site1.conf**

-Modifier son contenu depuis **nano site1.conf** de la sorte:

```
RedirectMatch ^/GestionFrais1/$ /GestionFrais1/site/
<VirtualHost *:80>
        # The ServerName directive sets the request scheme, hostname and port that
        # the server uses to identify itself. This is used when creating
        # redirection URLs. In the context of virtual hosts, the ServerName
        # specifies what hostname must appear in the request's Host: header to
        # match this virtual host. For the default virtual host (this file) this
        # value is not decisive as it is used as a last resort host regardless.
        # However, you must set it for any further virtual host explicitly.

        ServerName GestionFrais1.144.sio
        DirectoryIndex cAccueil.php
        ServerAdmin GestionFrais1@localhost
        DocumentRoot /var/www/html/GestionFrais1/site/

        # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
        # error, crit, alert, emerg.
        # It is also possible to configure the loglevel for particular
        # modules, e.g.
        #LogLevel info ssl:warn

        ErrorLog ${APACHE_LOG_DIR}/error.log
        CustomLog ${APACHE_LOG_DIR}/access.log combined

        # For most configuration files from conf-available/, which are
        # enabled or disabled at a global level, it is possible to
        # include a line for only one particular virtual host. For example the
        # following line enables the CGI configuration for this host only
        # after it has been globally disabled with "a2disconf".
        #Include conf-available/serve-cgi-bin.conf

        # A activer après quand tu as fait un certificat etc
        #SSLEngine on
        #SSLCertificateFile /etc/ssl/certs/apache-selfsigned.crt
        #SSLCertificateKeyFile /etc/ssl/private/apache-selfsigned.key
</VirtualHost>
```

Faire la même chose pour le site2 accessible qu'à l'utilisateur dev2 via GestionFrais2.144.sio :

```
RedirectMatch ^/GestionFrais2/$ /GestionFrais2/site/
<VirtualHost *:80>
        # The ServerName directive sets the request scheme, hostname and port that
        # the server uses to identify itself. This is used when creating
        # redirection URLs. In the context of virtual hosts, the ServerName
        # specifies what hostname must appear in the request's Host: header to
        # match this virtual host. For the default virtual host (this file) this
        # value is not decisive as it is used as a last resort host regardless.
        # However, you must set it for any further virtual host explicitly.

        ServerName GestionFrais2.144.sio
        DirectoryIndex cAccueil.php
        ServerAdmin GestionFrais2@localhost
        DocumentRoot /var/www/html/GestionFrais2/site
```

Remarque : La page d'accueil du site ne s'appelle pas « index.html » c'est pourquoi on doit rajouter « DirectoryIndex cAccueil.php ».

Nos sites seront donc accessibles via GestionFrais1.144.sio et via GestionFrais2.144.sio. Seulement, notre site n'a pas encore était installé et notre serveur DNS n'est pas encore configuré donc ça ne fonctionnera pas.

Le protocole SFTP permet quant à lui de partager des données entre deux machines de manière totalement chiffrée via l'application FileZilla, il va nous être utile pour mettre les fichiers de notre site sur le serveur. Pour l'installer il faut faire les commandes suivantes:

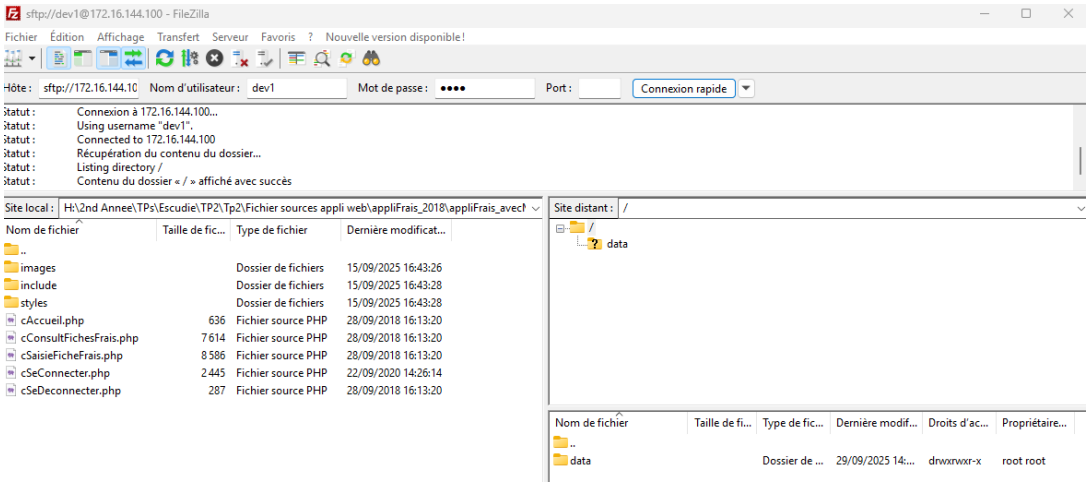
apt-get install openssl

groupadd sftp-users

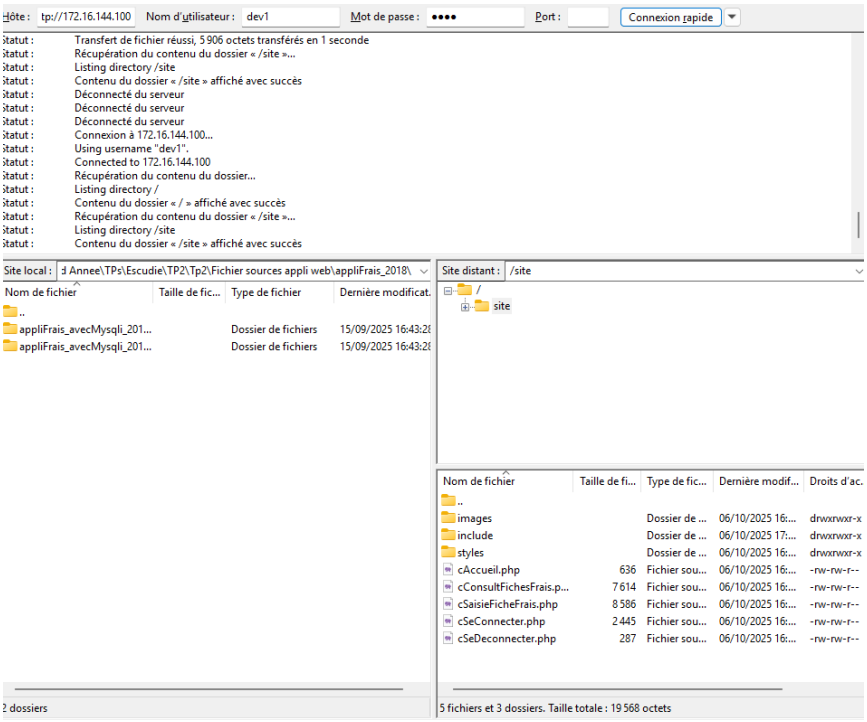
sudo a2enmod ssl

sudo systemctl restart apache2

Une fois ceci fait, on peut accéder au serveur web depuis FileZilla pour mettre les fichiers du serveur en toute sécurité, testons son fonctionnement avec l'utilisateur dev1. Pour cela ouvrez FileZilla et renseignez l'adresse IP du serveur et les logins de dev1 sur le port 22, si tout est bien configuré, lorsque vous cliquez sur « connexion rapide » cela devrait vous redirigez sur le dossier « GestionFrais1 » :



Sur le côté gauche on a accès aux dossiers de notre pc hôte et à gauche aux dossiers du serveur web. Pour mettre le site sur le serveur web il suffit donc de déplacer les fichiers du site du pc hôte vers le serveur :



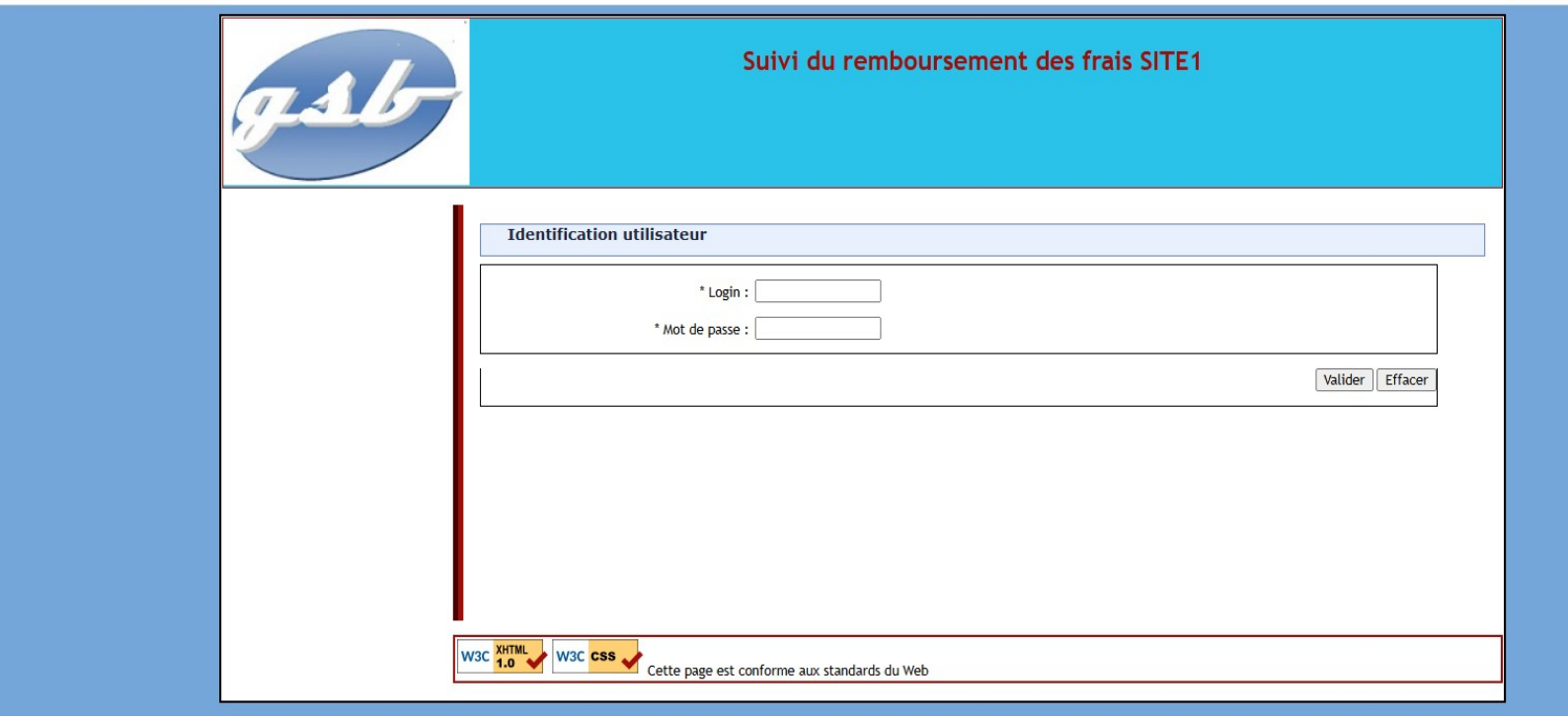
III. Publication du site en ligne

Nous allons maintenant rendre notre site accessible en ligne, premièrement il faut lié notre site à la base de donnée :
- Depuis le dossier « **/var/www/html/GestionFrais1/site/include** », entrez dans le fichier « **_bdGestionDonnees.lib.php** »
- Renseignez les informations de l'utilisateur dev1 de la BDD ainsi que celles de la BDD « GestionFrais1 » :

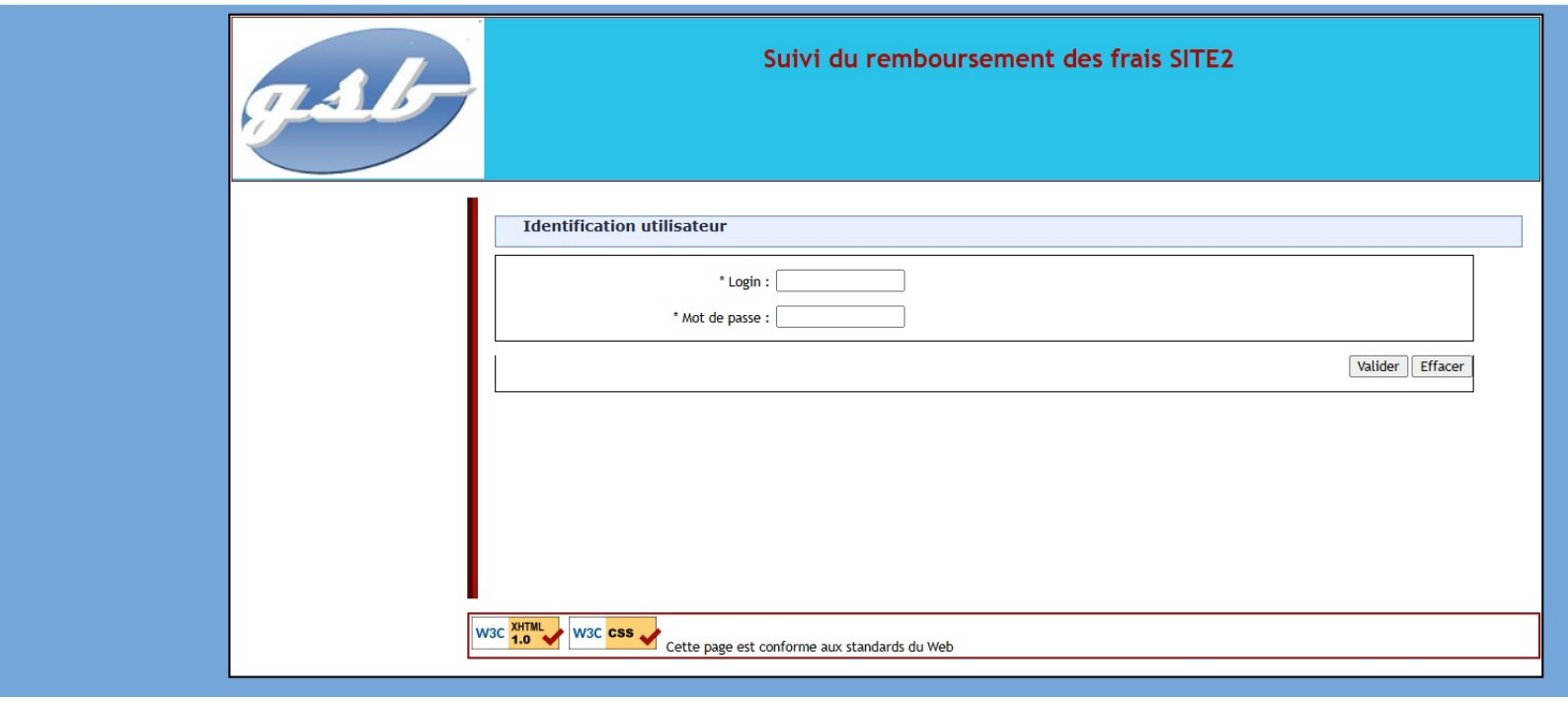
```php
<?php
/**
 * Regroupe les fonctions d'accès aux données.
 * @package default
 * @author Arthur Martin
 * @todo Fonctions retournant plusieurs lignes sont à réécrire.
 */

/**
 * Se connecte au serveur de données MySql.
 * Se connecte au serveur de données MySql à partir de valeurs
 * prédéfinies de connexion (hôte, compte utilisateur et mot de passe).
 * Retourne l'identifiant de connexion si succès obtenu, le booléen false
 * si problème de connexion.
 * @return resource identifiant de connexion
 */
function connecterServeurBD() {
    $hote = "localhost";
    $login = "dev1";
    $mdp = "Azerty31";
    return mysqli_connect($hote, $login, $mdp);
}

/**
 * Sélectionne (rend active) la base de données.
 * Sélectionne (rend active) la BD prédéfinie gsb_frais sur la connexion
 * identifiée par $idCnx. Retourne true si succès, false sinon.
 * @param resource $idCnx identifiant de connexion
 * @return boolean succès ou échec de sélection BD
 */
function activerBD($idCnx) {
    $bd = "GestionFrais1";
    $query = "SET CHARACTER SET utf8";
    // Modification du jeu de caractères de la connexion
    $res = mysqli_query($idCnx, $query);
    $ok = mysqli_select_db($idCnx, $bd);
    return $ok;
}
```

Faites la même chose pour le site2 avec les informations de l'utilisateur dev2 et de la BDD GestionFrais2.
- Enfin, fait la commande **sudo a2ensite site1.conf** et **site2.conf** pour activer les sites depuis le fichier .conf précédemment créé. Puis redémarrer le service Apache à l'aide de la commande **systemctl restart apache2**.
- Notre site devrait ainsi être accessible depuis « http://172.16.144.100/GestionFrais1 » ou « http://172.16.144.100/GestionFrais2 » :

Not secure 172.16.144.100/GestionFrais1/site/cSeConnecter.php



Not secure 172.16.144.100/GestionFrais2/site/cSeConnecter.php



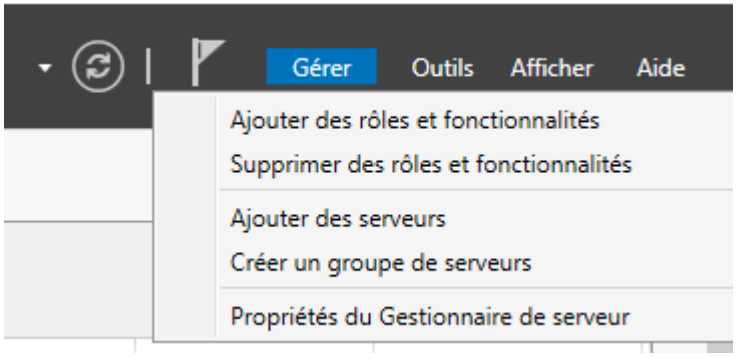
IV. Mise en place du DNS

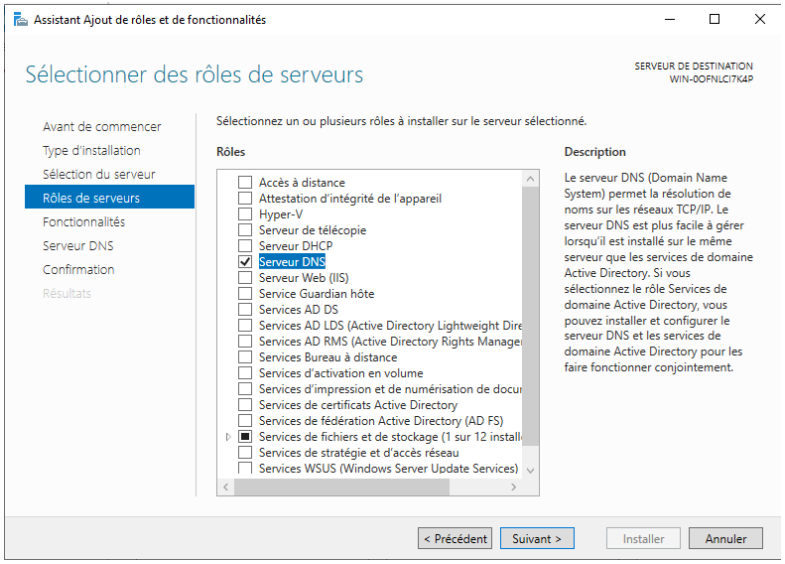
Le DNS (Domaine Name Service) va nous permet de résoudre le nom de domaine « GestionFraisX » en l'addresse IP de notre serveur web. Ainsi il sera possible d'accéder à nos deux sites respectif depuis « http://GestionFrais1.144.sio » et « http://GestionFrais2.144.sio ».

Notre DNS sera sous windows 2022 avec le configuration IP suivante :
→ **@IP** : 172.16.144.53
→ **Masque** : 255.255.0.0
→ **Passerelle** : 172.16.0.1
→ **DNS** : 172.16.0.100

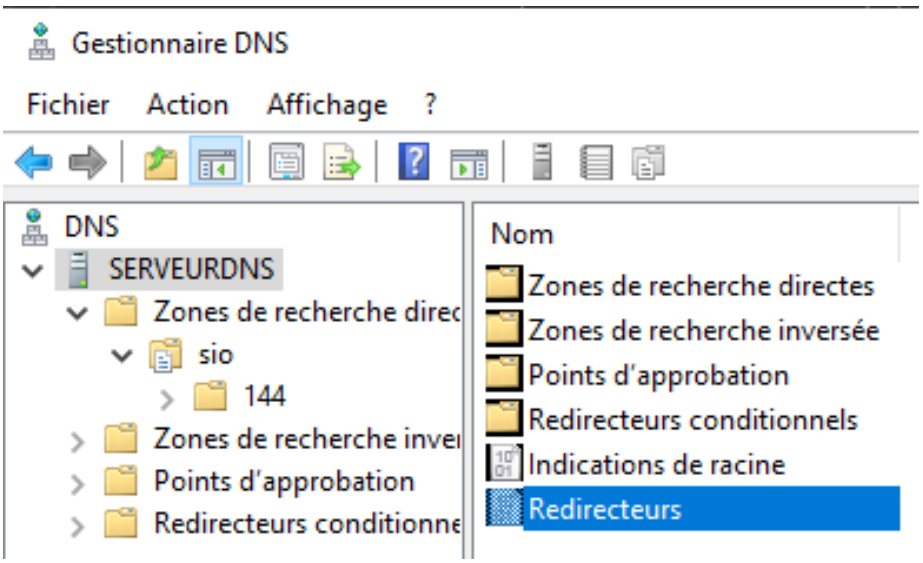
- Installation du service DNS

Dans un premier temps, rendez vous sur l'option "gérer" de votre gestionnaire de serveur puis cliquez sur "Ajouter des rôles et fonctionnalités". Continuez jusqu'à tomber sur une liste de services à installé puis cocher le service DNS :

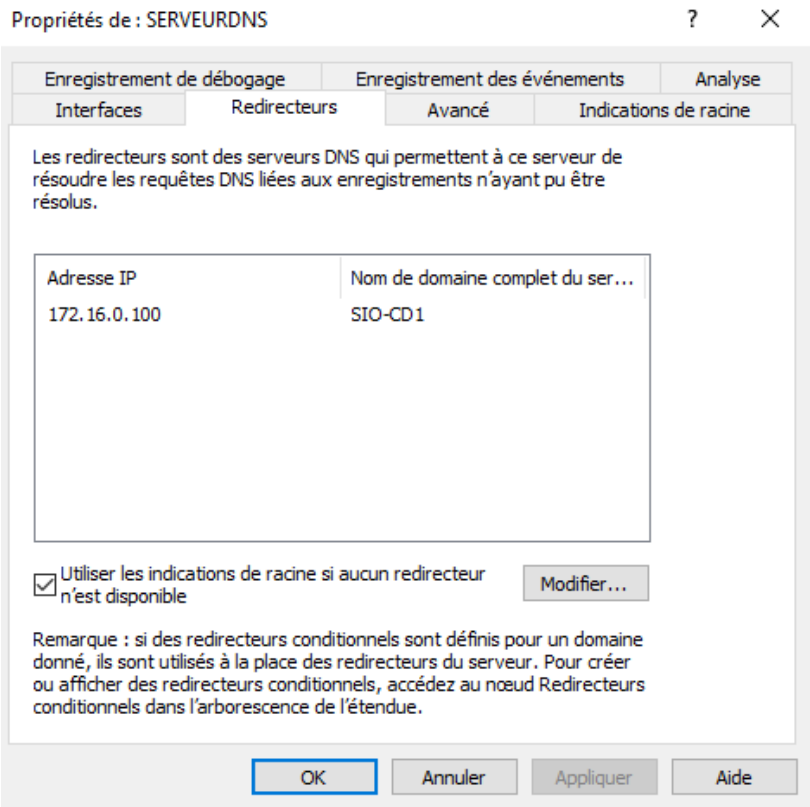




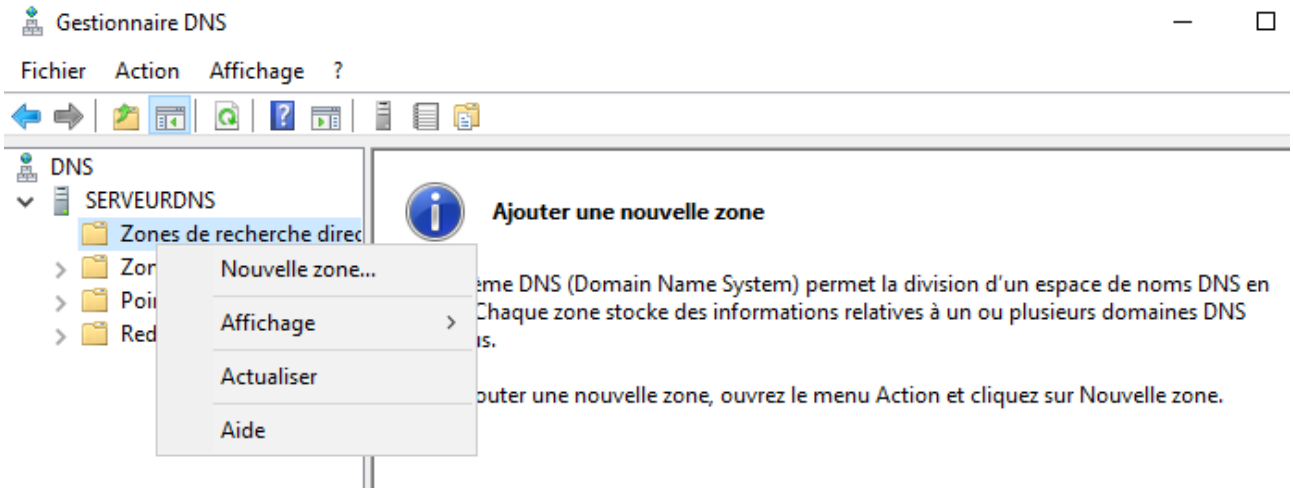
Faites suivant jusqu'à l'installation du service. Une fois installé, tapez « DNS » dans la barre de recherche Windows pour ouvrir la page de configuration de notre DNS. Une fois dessus, cliquez sur votre serveur DNS puis double cliquez sur « Redirecteurs » :



Un Redirecteur est un autre serveur DNS à qui on redirige les requêtes si notre propre DNS n'a pas pu la résoudre. Ce redirecteur possède lui aussi un redirecteur et ainsi de suite… Nous allons configurer le DNS du lycée en tant que redirecteur de notre DNS :



Ensuite, dans « zone de recherche directe » faites clique droit puis « Nouvelle zone » :

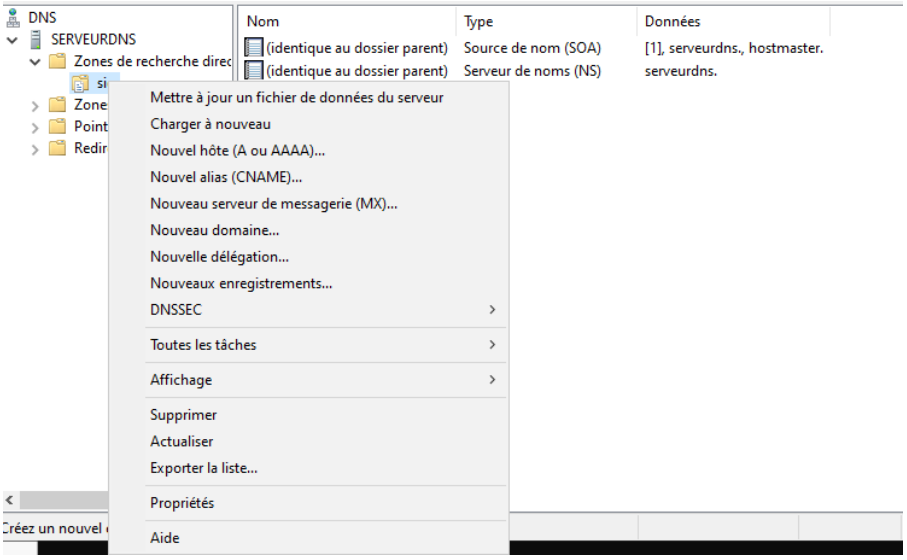


Configurez-la ainsi :

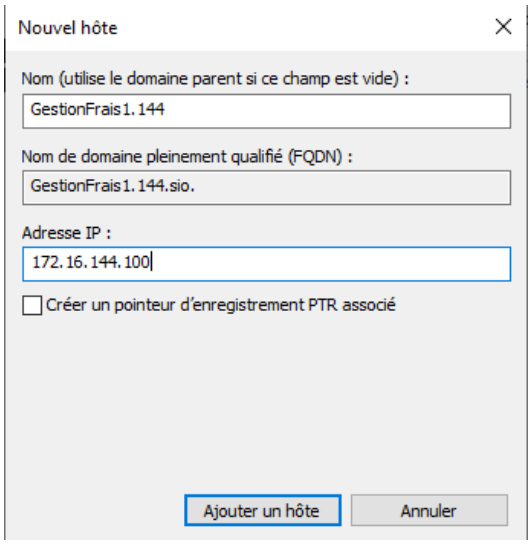
- Type de zone : « zone principale »
- Nom de la zone : « sio »
- Créer un nouveau fichier nommé : « sio.dns »
- Ne pas autoriser les mises à jour dynamiques

Cette zone à le rôle de zone DNS principale locale qui nous permettra de gérer des sous-domaines que nous allons créer dès maintenant.

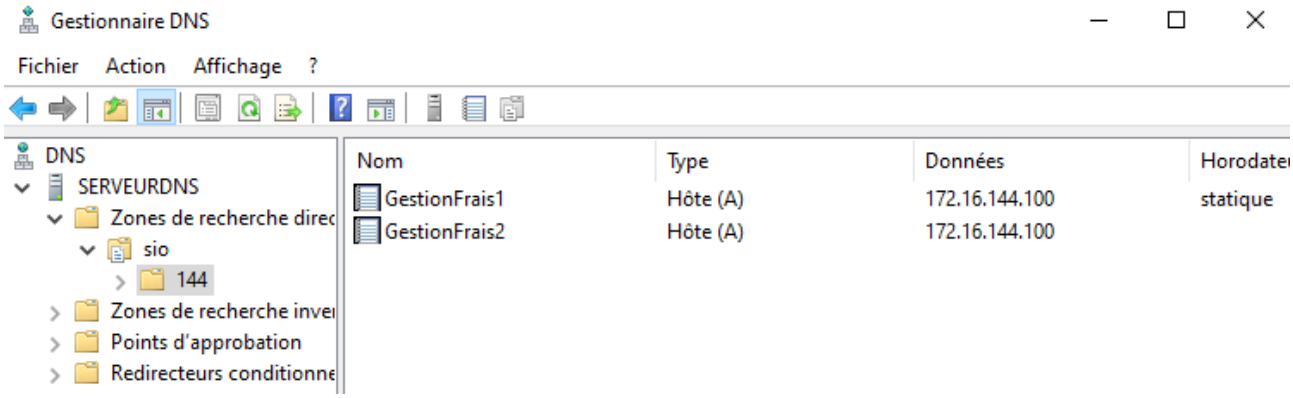
Une fois notre zone créée, faites clique droit dessus puis « Nouvelle hôte (A ou AAA) » :



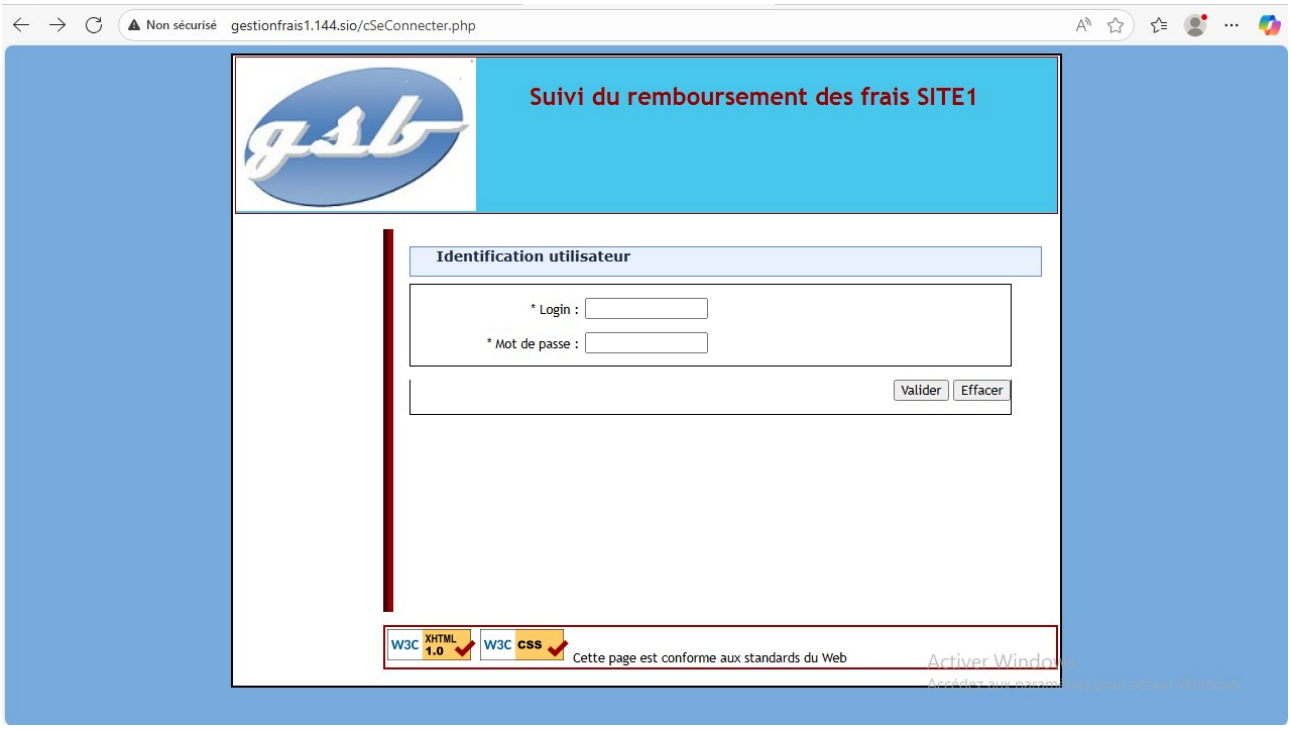
Pour chacun des deux sites, créez un nouvel hôte à son nom de la manière suivante :

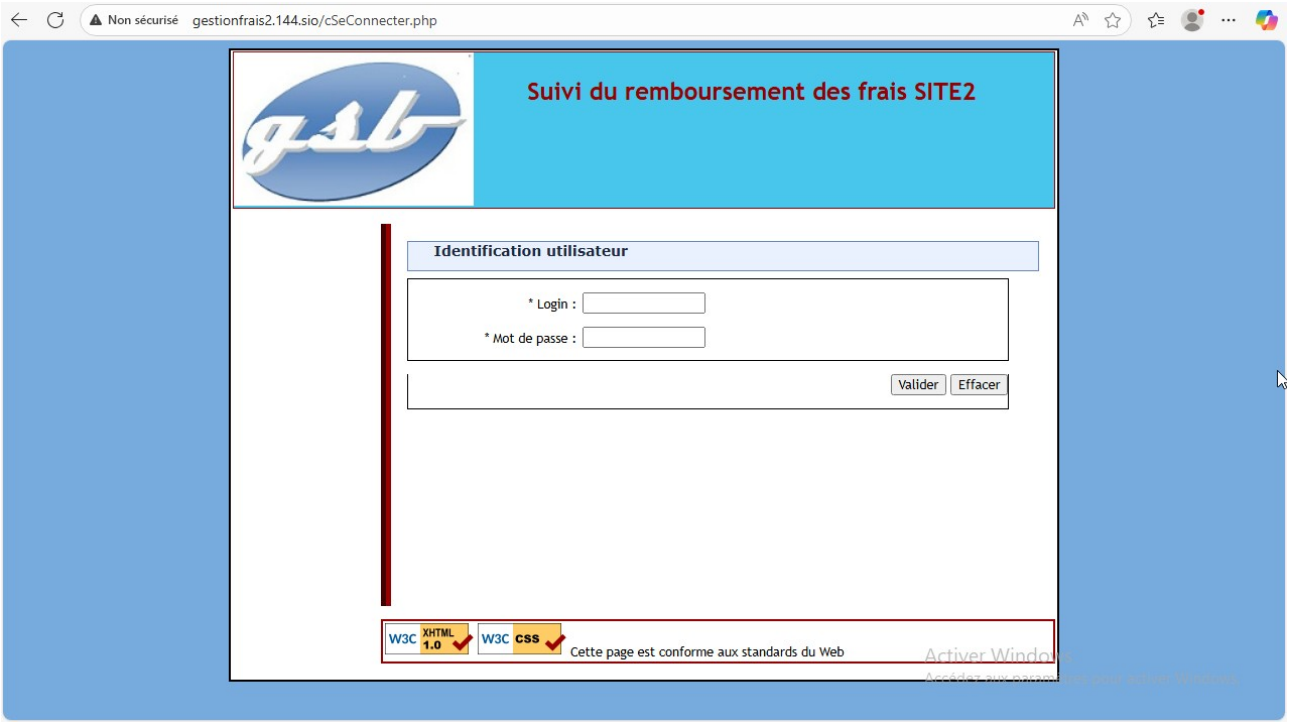


Ici, 144 sera notre sous-domaine et GestionFrais1 sera un hôte de ce sous-domaine. A la fin, on est censé avoir cette organisation :



Grâce à nos VirtualHost, l'accès aux sites depuis l'url « http://GestionFraisX.144.sio » est désormais possible. On peut vérifier cela via un PC de test ayant le DNS que l'on vient de créer.


Non sécurisé gestionfrais1.144.sio/cSeConnecter.php
Suivi du remboursement des frais SITE1
Identification utilisateur
* Login :
* Mot de passe :
Valider
Effacer
W3C XHTML 1.0
W3C CSS
Cette page est conforme aux standards du Web


Non sécurisé gestionfrais2.144.sio/cSeConnecter.php
Suivi du remboursement des frais SITE2
Identification utilisateur
* Login :
* Mot de passe :
Valider
Effacer
W3C XHTML 1.0
W3C CSS
Cette page est conforme aux standards du Web

V. Mise en place du protocole HTTPS

Rectification : Attention au nommage des fichiers car il y a sensibilité à la casse, pour des raisons de faciliter il faut tout écrire en minuscule, donc ne pas suivre le nommage du tp.

Actuellement nos sites web sont accessibles via le protocole HTTP depuis le port 80, or ce dernier n'est pas sécurisé c'est pourquoi nous allons mettre en place un serveur de certificat sous Debian13 afin d'accéder à nos serveur en HTTPS depuis le port 443. Pour commencer, voici la configuration IP de notre serveur de certificat :

```
  GNU nano 8.4                                                    /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug ens18
iface ens18 inet static
address 172.16.144.101
netmask 255.255.0.0
gateway 172.16.0.1
dns-nameserver 172.16.144.53
```

Ensuite , mettre à jour les sources.list comme effectué au début du TP puis faire les commandes **apt-get update** et **apt-get upgrade**.

Puis, nous avons besoin d'installer le service OpenSSL sur nos <u>deux serveurs</u> qui agi comme une sorte de boîte à outils de cryptographie permettant de créer des clés SSL/TLS. Pour cela il faut taper la commande **sudo apt install openssl -y**.

Pour accéder au fichier de configuration d'openssl il faut tapez la commande **nano etc/ssl/openssl.cnf**. Ce fichier nous permettra de retenir les informations nécessaire à la mise en place des certificats, le plus important à retenir est ici :

```
#####################################################################
[ ca ]
default_ca      = CA_default            # The default ca section

#####################################################################
[ CA_default ]

dir             = ./demoCA              # Where everything is kept
certs           = $dir/certs            # Where the issued certs are kept
crl_dir         = $dir/crl              # Where the issued crl are kept
database        = $dir/index.txt        # database index file.
#unique_subject = no                    # Set to 'no' to allow creation of
                                        # several certs with same subject.
new_certs_dir   = $dir/newcerts         # default place for new certs.

certificate     = $dir/cacert.pem       # The CA certificate
serial          = $dir/serial           # The current serial number
crlnumber       = $dir/crlnumber        # the current crl number
                                        # must be commented out to leave a V1 CRL
crl             = $dir/crl.pem          # The current CRL
private_key     = $dir/private/cakey.pem # The private key

x509_extensions = usr_cert              # The extensions to add to the cert

# Comment out the following two lines for the "traditional"
# (and highly broken) format.
name_opt        = ca_default            # Subject Name options
cert_opt        = ca_default            # Certificate field options

# Extension copying option: use with caution.
# copy_extensions = copy

# Extensions to add to a CRL. Note: Netscape communicator chokes on V2 CRLs
# so this is commented out by default to leave a V1 CRL.
# crlnumber must also be commented out to leave a V1 CRL.
# crl_extensions        = crl_ext

default_days    = 365                   # how long to certify for
default_crl_days= 30                    # how long before next CRL
default_md      = default               # use public key default MD
preserve        = no                    # keep passed DN ordering
```

→ certs accueille les certificats nouvellement créés à envoyer aux demandeurs
→ crl accueille les certificats révoqués
→ newcerts accueille une copie des certificats que l'organisation garde
→ private accueillera les clés privées
→ private/cakey.pem est la clé privée de l'autorité de certification
→ index.txt va contenir la liste des certificats que l'on générera
→ serial contient le numéro de série du prochain certificat créé : il est donc nécessaire de l'initialiser à « 1 »

ATTENTION : Pour la suite des étapes il faut modifier la variable « dir = ./demoCA» en « dir = /etc/ssl »

- Génération certificat auto-signé de l'autorité de certification

Remarque : Les commandes qui suivent doivent être exécutée dans /etc/ssl pour fonctionner.

Cette étape n'est à réalisé qu'une seule fois notre serveur de certificat, nous allons créer un clé privée CaKey.pem et lui appliquer des droits pour ensuite créer le certificat auto-signé CaCert.pem à partir de la clé privée. Pour cela, tape la commande **openssl genrsa -des3 -out private/cakey.pem 4096** (on se réfère au fichier précédent pour connaître le chemin) puis entrez une « pass phrase » (pour nous Azerty31). Ceci permet de créer un fichier cakey.pem contenant une clé privé de 4096 bits. Vérifiez que le fichier ait bien été créé :

```
root@Debian:/etc/ssl/private# ls -all
total 12
drwx------ 2 root root 4096 21 oct.  14:32 .
drwxr-xr-x 4 root root 4096 21 oct.  14:32 ..
-rw------- 1 root root 3422 20 oct.  19:36 caKey.pem
```

Si on rentre dans le fichier on aura accès à la clé privée :


```
-----BEGIN ENCRYPTED PRIVATE KEY-----
MIIJpDBWBgkqhkiG9w0BBQ0wSTAxBgkqhkiG9w0BBQwwJAQQigm+gAeYPNw5+M8e
omnu5QICCAAwDAYIKoZIhvcNAgkFADAUBggqhkiG9w0DBwQITayGNPlaUcwEgglI
49ykaBK58XHNsrYST9j+QtjCM5hIHkt1tHxeeouTIM6TV2FJTlvAbzk36BWHnuyP
pz85tjyLGuMNDPdGE+ChVPGlroZNvhO3Md9mNhrdlWnU8cXYNvpjLukyjjy1O4Q9H
S+Ka2INl/Chj0rIFO7SOJiCrJOKqM9YbMWJMxYstTedzKiHSmV9Gw6iTRb3BBkYC
3YNKdkHuH9TPXncQon1RSwtiX6FnV2g84WWtgkhF4CQR59hUu625nJiMHZMF2pAd
AjvaZVs2HDGShDrA/zWYAEAVKKXcNNCoMtpbLzTpc1FJvugsbKSFo9OXDm5cppkw
FKiCvmpIzUfQBgMbgjPuvnNcKj4Tks/E/pM/0o4j/m31luYslPfIA0Th+mfrY8hZ
PnMmEInakFWT3bpRlDXjd+8SbbQuD6YCrOymrOgsH0ctl/984UXzEmLoincqmEoJ
tL5pg0mbo+xXoaLnmL+RghSC1C40AFr0ZNSIK5GqxOWfwWK50tvwnTq8alrx5UKJ
4Tvxa5nfc/wDXTtdTbk7H5uN+uH1Zvo+ATOCxATlKgVWxXXMey3AnOduazQpMNqA
3EmcLBzkRr9PUjCp1Ll7oz5GSBIwiL9OHmL/CKkOoUgeBsxa7+YpUekCZ0KcUmAX
KgUriZd8xwV6WSHzoat5FjECnvcSlQTGiFleOBsBtvknw1Ky4Yk9eDKtdYthPqnw
EEb3htsuXE9S5T/U5PRsn6pjUhTw6mk/JgpD17H+La9ddZB1qn0NBy/D7aMxO6lT
UBRFmZA2Tni8hSZw37QPryG7xne/DW1opPL37JobWsyER+LOho7RUfNMKcGgpqpf
v9ATXAI7Ze/B7IWdgRg7Mj2fwNhjp4X8hyC3DoFubMoMyFsQXHcXeshixIj9fe+O
ErAyVDN+X76Lt3S5bjF04k4/sFr8CBg5QxDYlfk5wABF8jNhi3sFEMpuT7SPhdD6
pbiRPzJ/f6MRglpl+rxXsJlcYzJyGvj54UA/6x7INa/VtONn+jwjD5TISJClRulE
9bm8eykYg8EQfbRbTCsM43+PhA9arMMfSX0e6BiBBwXz/DC79K4Nwrx5kzMu5EDh
kxuPjG7S+p1PTwvqEmQZHUvwXtv5jwTy7AolhqaGtStZQR9/JiaDz7smR6Z03d6m
hx8rH1kN60LpITvyDMgaGnklPvqWvl438vvji1Gb7xOF/4S6dZv3AJSXnV/bdfWA
Zeq6fOFTshL6hNT5rofkeWRDhTsT1HNVY3jm6Sxy+AUUQlIxk9k/hLpSLRNq+vK9
Il2k436AOpVs9WAlAon4lwyqOfMM6THNHbxu9CMB7Sh8oFB5ne6yHfHtn1IHLfnB
qMzKeaFD/KpI3iBmTI/t4oDXMemWomMYVGc14J2V2iPCpCPjiQnuemlETp/lyUwH
IKdz+I7VeMS2V4Rw2KHe4TZgB9Yt9FFJIOKWZUQlD2sNL6vxqKkncNnVFyHqTHC3
6Bn3yIRXiqVv26G1M4RgKnFktaicrkGWsPtj3eiqAGN0ifs2pMgjp/rZ3KoIH4ea
g+V2tn8rFBZ7jI7o0pBTPxFCbOV017qNtT24IRR6H1TNpz1dAq0eDrrNoCoy5ppa
L2YV9Cjdbr5YlDKACcJx8S4kLEKIysK+5Gy1RlK6fAB/qAnle9ocIEuKZ07ZWwfp
79i3HwNbz+2S8u0bXWhWl8v8db4qHHongYgncA2TWldhxVz5xI+fPY6bb12Tep7T
irbljwvvgtntCnCt5DYUCwYIvJOtlwBk5XCQTxXbQN5/Imw0/C7ZRS5rUiW6O5o9
QRfcVv9QFjQtCxTzj1kJ/3RF1QbLjbPxEn07D2pt+EdbpStP6Kbpn4tQwDohhcTE
VXo9r3Vb435t06aAedlqUJ2NSlDyJpomk+++NBpq+zXWbj6tVB9jVZGZ8NJY3PhA
hlPFA2P/GHCYJl9RvaHek43Owf6axHKs0TSOOQ0TesJzinWzpCWnpOgo0qjQWSxf
qAiZ/O4ryZqHO3ijafnYBBM1fjKFlMyEeN5voSFr2kca1gZOfXap31TehG4lsyKX
IpX7lCooJTZuBOmSTtjGpkjj8rje4DCyQdJmY20kqkGMLXmqYviLVa4ppUGyMZ2D
UlePvpWUMkvuZ4lr96Dwl20iyc5BOIN86ZRIflYPHcpg/qnAXYtvTSBVKP6czery
Vmo597573aS3qxk2J0MJb0iZDX3nNi0lP9q5N45QtgsX3n9CgRIII22wB/B9o6Nr0
q078XJczzeUxyLGr8jF4K45bybR2KEkjL05hm3DvEskdn5HYLbtP6C95iWNnxAiX
ZgU9vZyJxZ1xWOwD3QKhoEjibIR6zXqgUjIprxzfb0gdzFJ3Qy1Fnf5KEHpCoprS
mZpM5wU+Hiadg8YmIZg3vmkk83L8mGbMJ6VW0Ck87eKDdaPa5TDJ5Bd/fR5ljlu/
6VX0zlu5cSDEi18+3xo3tmfFwP2kyR8kmdpLN5a5i9GgHktDpGBG/GPoLzK4tzJm
sBHtc+0cFwMIxeF112MmUJ57d/4qs5+oUUIjiOYzRyJqm7hOy+DMXDYRBApkRM6X
+9Gix35kPy5MWbVl4wBx0uR/gBe35d2X/4oDoYkvKhwc9+KzQuWH93TAgGB9VjyB
VzC3E/swj7yI81+Yc+grfMiSwTB80SKy4stu3Pxv7ikXYtUGqaqzlSlELEzTyxG5
BJozVt1J6/KrLopguJwU0WKBkZ53+jDImW9aa7mnJBJdM3dUcTP4lrH+/TjMceX+
```

Ensuite, nous allons générer notre certificat auto-signé depuis la commande :

openssl req -new -x509 -days durée_en nbre_jours -key chemin/cakey.pem -out chemin/nom_certificat.pem

- *req -new* : demande d'un nouveau certificat
- *x509* : le certificat demandé est auto-signé
- *days durée_en nbre_jours* : la durée de validité du certificat sera du nombre de jours spécifié (pour nous 5 ans donc 1825 jours)
- *key chemin/cakey.pem* : le chemin vers la clé privée à partir de laquelle sera généré le certificat contenant la clé publique correspondante.
- *out chemin/nom_certificat.pem* : création d'un fichier de nom nom_certificat.pem (qui sera le certificat auto-signé de l'autorité de certification) dans /etc/ssl (voir fichier /etc/ssl/openssl.conf)

La commande final sera donc :

openssl req -new -x509 -days 1825 -key private/cakey.pem -out /etc/ssl/cacert.pem

A l'exécution de la commande la « pass phrase » de la clé privée vous sera demandez, ensuite vous pouvez commencer à renseigner les informations du certificat auto-signé tel que le nom du pays, le nom de l'organisation, l'adresse e-mail…

```
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:Europe
Locality Name (eg, city) []:Paris
Organization Name (eg, company) [Internet Widgits Pty Ltd]:GSB
Organizational Unit Name (eg, section) []:Laboratoire France
Common Name (e.g. server FQDN or YOUR name) []:CA GSB France
Email Address []:cagsbparis@swiss-galaxy.com
```

Ce certificat va permettre de vérifier l'authenticité et la validité du certificat du serveur Web.

- <u>Demande d'un certificat auto-signé sur le serveur web</u>

Remarque : Les étapes qui suivent devront être effectuées deux fois pour créer deux certificats, un pour le site GestionFrais1 et un autre pour le site GestionFrais2. Pour faciliter les explications, l'utilisation de « X » dans les commandes désignera soit le site 1 ou 2.

Sur notre serveur web cette fois-ci, nous allons effectuer les mêmes étapes que précédemment sauf pour la création de la clé privé qui ne sera pas protégée par une « pass phrase » (on enlève donc l'option -des3 sur la commande). La commande pour créer nos clés privées sera donc **openssl genrsa -out private/servWebXKey.pem 4096**

Maintenant, à partir de ces clés privées, nous allons faire une demande de certificat à notre serveur de certificat, on appel cette demande un « CRS ». Pour effectuer ce CRS il faut taper la commande suivante pour chaque sites :

openssl req -new -key private/servWebXKey.pem -out /etc/ssl/servWebXDem.pem

Remplir le formulaire tel quel puis faire de même pour le second site :

```
root@Debian:/etc/ssl# openssl req -new -key private/servWeb1Key.pem -out /etc/ssl/servWeb1Dem.pem
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:Europe
Locality Name (eg, city) []:Paris
Organization Name (eg, company) [Internet Widgits Pty Ltd]:GSB
Organizational Unit Name (eg, section) []:Laboratoire France
Common Name (e.g. server FQDN or YOUR name) []:gestionfrais1.144.sio
Email Address []:gsbparis@swiss-galaxy.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:.
An optional company name []:.
```

Attention sur le Common Name (CN) : celui-ci doit correspondre exactement au nom pleinement qualifié (nom DNS : celui que vous allez saisir dans l'URL de votre serveur qui correspond aussi à la directive ServerName dans le fichier de configuration d'Apache).

- Production du certificat signé par l’autorité de certification

Cette signature et la production du certificat se réalisent avec la commande suivante :

openssl ca -policy policy_anything -out chemin/nom_du_certificat.pem -infiles chemin/nom_fichier_dem.pem

- ca : c’est l'autorité de certification qui agit.
- policy policy_anything : c’est la clause policy_anything du fichier de configuration openssl.cnf qui sera utilisée. Dans notre cas seul le « Common Name » sera testé.
- out chemin/nom_du_certificat.pem : le certificat généré et à envoyer
- infiles chemin/nom_fichier_dem.pem : le fichier de demande de certificat utilisé pour générer le certificat.

Notre commande à effectuer sur le serveur de certificat est donc :

openssl ca -policy policy_anything -out /servWebXCert.pem -infiles /servWebXDem.pem

Rentrez à nouveau la « pass phrase » de cakey.pem

Dû à de nombreux problèmes dont je n’ai pas la solution, je ne peux pas allez plus loin pour cette étape. Cependant, je vais expliquer ce que nous étions censé faire si cela avait marché.

- Configuration d’APACHE2 avec mod_ssl

Rendez vous sur les virtual host des site 1 et deux et modifiez le port 80 en 443 :

```
<VirtualHost *:433>
        # The ServerName directive sets the request scheme, hostname and port that
        # the server uses to identify itself. This is used when creating
        # redirection URLs. In the context of virtual hosts, the ServerName
        # specifies what hostname must appear in the request's Host: header to
        # match this virtual host. For the default virtual host (this file) this
        # value is not decisive as it is used as a last resort host regardless.
        # However, you must set it for any further virtual host explicitly.

        ServerName GestionFrais1.144.sio
        DirectoryIndex cAccueil.php
        ServerAdmin GestionFrais1@localhost
        DocumentRoot /var/www/html/GestionFrais1/site/
```

Ensuite, dans le même fichier, nous devons renseigner les directives suivante entre les balises <VirtualHost @IP:port> et </VirtualHost> :

SSLEngine on
SSLCertificateFile chemin_vers_le_certificat_du_serveur_web
SSLCertificateKeyFile chemin_vers_la_clé_privé_du_serveur_web

Si tout est bien effectué, on peut désormais accéder à nos sites en https. Cependant la page s’affichera quand même en « non sécurisée » car c’est un certificat auto-signé.